

Partie 3

Le modèle mémoire de Java

La mémoire

Représentation primaire de la mémoire

Premier niveau d'abstraction :

- Mémoire de l'ordinateur = **tableau** d'une seule colonne,
- chaque case a une **adresse** (indice de ligne),
- (en Java, les **références** sont des adresses,)
- contenu des cases : **mots binaires**, représentant chacun
 - une donnée primitive,
 - ou une adresse.

Note

Toutes les cases ne sont pas utilisables, mais cela est géré par le compilateur, en collaboration avec le système d'exploitation.

Représentation secondaire de la mémoire

Deuxième niveau d'abstraction, en trois zones :

- la **zone statique**, pour
 - les instructions du programme,
 - les valeurs des constantes, les littéraux,
- la **pile**, pour
 - les variables et les paramètres des méthodes,
 - mémoire à court terme,
- le **tas**, pour
 - les objets et leur propriétés, les tableaux,
 - mémoire à long terme,

On y ajoute les **registres**, très peu nombreux, non directement utilisables par le programmeur.

La zone des instructions

Zone statique :

- chargée au lancement, **non modifiée** durant l'exécution ;
- contient les **instructions compilées** du programme, groupées par méthodes ;
- contient aussi les constantes et littéraux, en particulier les chaînes de caractères ;
- contient pour chaque classe, la **table des méthodes** (avec une référence vers la première instruction de chacune).

Statique = **ne change pas**

La zone du tas

Le **tas** (*heap*)

- contient les **objets** et **tableaux** créés **durant l'exécution** du programme ;
- chaque objet/tableau occupe un **bloc**, constitué de **cases consécutives** ;
- les propriétés des objets ou le contenu des cases des tableaux sont directement stockés dans le bloc :
 - par leur valeur (types primitifs),
 - par leur référence (types références).
- **Référence** = adresse du premier emplacement du bloc.

Note

En Java, les blocs du tas, une fois alloués, existent indéfiniment.

La zone de la pile

La **pile** (*stack*)

- contient les variables et les paramètres de chaque méthode en cours ;
- organisée en **couches**, une couche par méthode en cours d'exécution ;
- chaque appel de méthode crée une nouvelle couche, qui contiendra **this**, les paramètres et les variables de la méthodes, et qui masque les autres couches ;
- la couche est détruite lorsque l'appel de méthode retourne (avec **return**).

Note

Comportement dynamique (apparition et disparition des valeurs).

Représentation des objets dans le tas

Un objet dans le tas = un **bloc** de cases consécutives.

- Une case par propriété de l'objet, contenant :
 - une valeur (type primitif),
 - une référence (type référence).
- Une case pour identifier la **classe** de l'objet, contenant une référence vers la **table des méthodes** de la classe dans la zone d'instruction.

Polymorphisme

L'objet porte ses méthodes ! Ainsi l'appel d'une même méthode sur deux objets différents peut exécuter deux blocs d'instructions différents.

L'effet des instructions

La déclaration de variable

Dans une méthode, la **déclaration d'une variable** provoque :

- 1 l'ajout de la variable dans la **couche active de la pile**,
- 2 l'initialisation de cette variable.

L'affectation

Résolution de l'affectation :

- ① évaluation de la *left-value* $\longrightarrow$ emplacement mémoire m :
 - variable, paramètre de la méthode : $m \in$ couche active de la pile,
 - propriété d'un objet, d'une classe, cellule d'un tableau : $m \in$ tas.
- ② évaluation de la *right-value* $\longrightarrow$ valeur v :
 - une valeur (type primitif),
 - ou une référence (type référence).
- ③ remplacement du contenu de la case m par v .

La construction d'un objet

Résolution de la création d'un objet avec `new` :

- ① allocation d'un bloc du `tas`,
- ② initialisation des propriétés de l'objet,
- ③ appel du constructeur (*cf* appel de méthode),
- ④ retourne la référence du nouvel objet = adresse du bloc.

L'appel de méthode

Résolution d'un appel de méthode :

- ① évaluation de l'objet appelant la méthode,
- ② recherche de la première instruction de la méthode (en utilisant la table des méthodes de l'objet dans la **zone d'instructions**),
- ③ évaluation des arguments de la méthode,
- ④ sauvegarde du pointeur d'instruction dans la couche active de la **pile**,
- ⑤ ajout d'une couche dans la **pile**, avec **this** contenant la référence de l'objet, et une case par paramètre, contenant les valeurs des arguments,
- ⑥ saut du registre **pointeur d'instruction** vers la première instruction de la méthode.

La méthode est ensuite évaluée jusqu'au retour.

Le retour de méthode

Résolution de **return** :

- ① évaluation de l'expression de la valeur de retour (dans un **registre**),
- ② suppression de la couche supérieure de la **pile**,
- ③ restauration du **pointeur d'instruction**, stocké lors de l'appel de méthode, provoquant un **saut** vers l'instruction ayant appelé la méthode qui vient de retourner.

La valeur de retour est disponible dans son **registre**, pour terminer l'instruction contenant l'appel de méthode.

Récapitulatif

Modèle d'évaluation

- **Modèle de la mémoire**, abstraction en 3 zones :
 - zone statique,
 - pile,
 - tas.
- **Modèle d'évaluation**, expliquant :
 - chaque instruction,
 - les appels de méthodes, avec la pile,
 - les structures de contrôle,
 - le polymorphisme objet : la méthode exécutée est celle de l'objet, déterminée à sa création.

Intérêt du modèle

Les aspects techniques doivent être compris : important pour

- comprendre le comportement d'un programme,
- interpréter et corriger des erreurs d'un programme,
- améliorer les performances d'un programme.

D'un point de vue fondamental, liens avec l'architecture, les systèmes d'exploitation, la compilation, la théorie des langages de programmation.

Perspective

Ces connaissances sont essentielles pour devenir un programmeur compétent.